動的解析ログの局所・大局的特徴の画像化とCNNスタッキングによるマルウェア検知

横田 フェルナンド拓哉[1,a)]　青木 茂樹[1,b)]　宮本 貴朗[1]

概要：動的解析ログを用いたマルウェア検知では，API 名・頻度・順序・引数を統合利用する従来手法が存在するものの，時系列上の配列位置への直接依存により，ダミー API 挿入などの順序撹乱（位置ズレ）に弱い課題がある．そこで本稿では，動的解析ログの特徴パターン（2-gram や単語）に固定インデックスを与え，出現有無や頻度を描画した固定座標の画像表現へ抽象化することで，位置ズレの影響を回避する手法を提案する．さらに，局所的な順序関係と大局的な出現頻度を表す複数画像から CNN で抽出した中間特徴を，アンサンブル学習の一種であるスタッキングにより統合し，表現の相補性を活用する．実験の結果，提案手法が 1D 特徴量に基づく手法や単体画像モデルを上回る検知性能を達成し，1 ファミリあたりの学習データが限られた状況下でも安定して動作することを確認した．

キーワード：マルウェア検知，動的解析ログ，画像化，CNN，スタッキング

Malware Detection from Dynamic Analysis Logs Using Image Encoding of Local and Global Features with Feature-Level CNN Stacking

FERNANDO TAKUYA YOKOTA[1,a)]　SHIGEKI AOKI[1,b)]　TAKAO MIYAMOTO[1]

Abstract: Although conventional malware detection methods using dynamic analysis logs comprehensively utilize API names, frequencies, sequences, and arguments, their direct dependence on time-series positions makes them vulnerable to sequence perturbations such as dummy API insertions. To mitigate this issue, we propose a method that abstracts log features (e.g., 2-grams and words) into fixed-coordinate image representations by assigning them fixed indices and rendering their presence or frequencies as pixel values. Furthermore, intermediate features extracted by convolutional neural networks (CNNs) from multiple images representing local sequential relationships and global occurrence frequencies are integrated using stacking, an ensemble learning technique, to exploit feature complementarity. Experimental results demonstrate that the proposed method outperforms 1D feature-based methods and single-image models, maintaining stable detection performance even with limited training data per malware family.

Keywords: malware detection, dynamic analysis logs, image encoding, CNN, stacking

1. はじめに

近年，情報通信技術の発展に伴い，サイバー犯罪の脅威は拡大の一途をたどり，深刻な社会問題となっている．攻撃者が用いるマルウェアは，検知を回避するためにコードを暗号化・変形する難読化技術や，多様な回避技術を用いて高度に進化している．こうした技術の影響を強く受けるのが，プログラムを実行せずに解析する静的解析や，既知の特徴と照合するパターンマッチング方式である．これらは軽量かつ高速である一方，未知のマルウェアや，実行のたびに自身を変化させるポリモーフィック型・メタモーフィック型のマルウェアに対しては十分な検知能力を発揮

[1] 大阪公立大学大学院情報学研究科
Graduate School of Informatics, Osaka Metropolitan University
[a)] sf22420q@st.omu.ac.jp
[b)] aoki@omu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

できない．

このため，マルウェアを隔離環境で実行し，実行時の挙動を直接観測する動的解析が注目されている．動的解析では，実行中に呼び出される API や OS への命令，通信，ファイル操作といったログを収集し，その振る舞いに基づいて悪性・良性を判断できるため，難読化の影響を受けにくく，マルウェアの本質的な機能を捉えられる．これに伴い，収集された動的解析ログに基づくマルウェア検知の自動化や高度化を目指し，機械学習や深層学習を適用する手法が多数提案されている．文献 [1] では，API 呼び出しの出現頻度を特徴量として Random Forest により分類する手法が提案されているが，呼び出し順序や引数の値を考慮しないため，良性検体と類似した API を使用しながら，引数や順序を巧妙に操作するマルウェアの検知に課題が残る．また，文献 [2] は API 名とそのカテゴリを組にした要素の遷移パターンを画像化し CNN で分類する手法を提案しているが，引数情報を含まないため，ファイル操作先や通信先といった具体的な挙動の差異に基づく識別には課題が残る．さらに，文献 [3] は API 名と引数の出現頻度に基づくグレースケール画像化手法を提案し CNN で分類しているが，実行順序が考慮されないため，順序関係が攻撃手順の本質となる場面への対応が課題となる．

以上のように，API 名・引数・カテゴリという基本要素，およびそれらが形成する「順序」と「頻度」の情報を包含して組み合わせた研究は少ない．加えて，時系列上の配列位置に直接依存する手法では，ダミー API の挿入などによる順序撹乱（位置ズレ）の影響を受けやすい．また，攻撃者は目的達成のために異なる API を使い分けるため，API 名そのものに依存した表現では未知亜種への対応が課題となる．

そこで本研究では，動的解析ログに含まれる API 名・引数・カテゴリの 3 種類の基本要素を統合的に扱うための画像化手法を新たに設計し，さらに CNN による特徴抽出とスタッキングを組み合わせることで，検体の良性・悪性を識別する 2 値分類手法を提案する．

提案手法では，抽出した特徴パターンに固定のインデックスを与えて画素位置とし，出現の有無や頻度を画素値として描画する．各特徴に対応する画素位置は検体によらず一定であるため，ログ全体の時系列上の絶対的な配列位置（出現順序のインデックス）への直接依存を回避し，撹乱が生じても悪性挙動の特徴パターンを固定座標上の有無・頻度として空間配置・抽象化できる．さらに，性質の異なる複数の画像から CNN で中間層特徴を個別に抽出し，スタッキングにより統合することで，良性・悪性の高精度な識別を行う．

以下，第 2 節で関連研究，第 3 節で提案手法，第 4 節で実験と考察を述べ，第 5 節でまとめとする．

2. 関連研究

本節では，動的解析ログを対象とした先行研究 [1], [2], [3], [4] と，本研究の識別部で用いるスタッキングに関する研究 [5], [6], [7] について述べる．

文献 [1] は，動的解析ログから抽出した API 呼び出しの出現頻度を特徴ベクトルとし，Random Forest で分類する手法を提案している．大規模な検体に対して高い精度を達成している一方，API の呼び出し順序や引数を考慮しないため，それらを巧妙に操作するマルウェアの特徴を十分に捉えきれない．

文献 [2] は，API 名とカテゴリの組を要素とする遷移パターンを n-gram として抽出・画像化し，CNN で分類する手法を提案している．API 名にカテゴリを付加することで，API 識別子の曖昧性を解消し，操作種別を特徴に反映している．その後，特徴選択によって寄与度の高い n-gram を選び，各 n-gram の有無を画素値とする二値画像を生成する．一方，カテゴリは API 名の補助として付加されるにとどまり，API 名自体を抽象化したパターンの抽出は行われない．また，引数情報を利用しないため，ファイルパスや通信先など具体的な操作対象の識別には課題が残る．

文献 [3] は，API 名と引数を結合した語の出現回数を画素値へ変換し，グレースケール画像として CNN で分類する手法を提案している．引数を含めることで，ファイル名や通信先などの操作対象を反映できる一方，出現回数のみを用いるため実行順序は保持されない．文献 [4] も同様に，API 呼び出し列を言語処理の枠組みで扱っているが，大局的な統計傾向やテキスト表現を中心とするアプローチでは，実行順序の密接な依存関係を捉えきれず，ダウンローダやドロッパのように特定の攻撃手順（呼び出し順序）が本質となるマルウェアの検知には課題が残る．

一方，単一モデルの性能向上にとどまらず，複数モデルの予測結果を統合することで全体の汎化性能を高めるスタッキングアプローチも注目されている．スタッキングをマルウェア検知に適用した研究として，文献 [5] は，PE ヘッダの静的特徴に対し，全結合ネットワークと 1 次元 CNN をベースモデル，決定木モデルをメタモデルとして用いている．文献 [6] は，メモリダンプ由来の特徴に対して複数の機械学習モデルとニューラルネットワークを組み合わせている．また，文献 [7] は，Android アプリケーションの dex ファイルを画像化し，複数の CNN を SVM で統合している点で，複数モデルの出力を統合する本研究の構成と共通性を有する．

しかし，既存のスタッキング手法はいずれも単一の入力表現（同一の画像や特徴ベクトル）を前提としており，ベースモデルの多様性をモデル構造やアルゴリズムの違いのみに依存している．そのため，単一の表現形式が持つ偏りや

© 2026 Information Processing Society of Japan

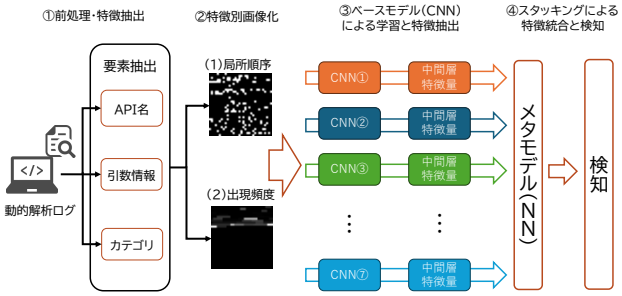


図 1　提案手法の全体概要
Fig. 1　Overview of the proposed method.

情報欠損の影響を直接受ける課題がある．これに対し本研究では，動的解析ログから「順序」と「頻度」という性質の異なる複数の画像表現を抽出し，入力データそのものの多様性を確保する．これにより，モデル構造のみに頼ることなく，特徴表現間の相補性を直接活用した統合検知を実現する点に本研究の特徴がある．

3. 提案手法

提案手法の処理の流れを図 1 に示す．本手法は，(1) 前処理・特徴抽出，(2) 特徴別画像化，(3) ベースモデル（CNN）による学習と特徴抽出，(4) スタッキングによる特徴統合と最終検知，の 4 段階で構成されている．まず，動的解析ログから API 名・引数・カテゴリの 3 種類の基本要素を抽出する．次に，これらの要素を組み合わせて複数の画像パターンを生成する．続いて，各画像パターンを CNN で個別に学習し，中間層の特徴ベクトルを抽出する．最後に，これらの特徴ベクトルをスタッキングで統合し，メタモデルにより良性・悪性を識別する．

3.1 前処理と特徴抽出

本研究では，ソフトウェアを Windows 環境で実行した際に記録された動的解析ログを利用する．ログの中には検体を実行した際に記録された API 呼び出し名，引数，関連するモジュール名，各種ハンドル情報など，詳細な実行ログが含まれている．まず，提供されたログから以下の 2 種類の基本要素を抽出する．

(1) **API (FunctionName)**：呼び出された OS・API の名称
(2) **引数 (Parameter List)**：API が受け取った値やパスなどの実行対象情報

さらに，API 名をそのまま使用すると攻撃者による亜種生成（API 名変更）に弱いため，API の機能的性質に基づくカテゴリを各 API に付与する．カテゴリを用いる先行研究 [2] では動的解析ツールが出力する分類をそのまま利用しているが，本研究で用いるログはこれとは異なるツールで収集されており，同等の分類情報が付与されていない．

表 1　API カテゴリの定義
Table 1　Definition of API categories.

カテゴリ	API 名
Resource Creation	NtCreateFile, NtCreateKey, NtCreateUserProcess
Resource Open	NtOpenFile, NtOpenKey, NtOpenProcess
Write / Modification	NtWriteFile, NtWriteVirtualMemory, NtSetValueKey
Query Information	NtQueryDirectoryFile, NtQuerySystemInformation
Read Operation	NtReadFile, NtReadVirtualMemory
Map / IO Control	NtMapViewOfSection, NtDeviceIoControlFile
Image / Driver Load	LdrLoadDll, NtLoadDriver
Enumeration	NtEnumerateKey, NtEnumerateValueKey
Memory Management	NtAllocateVirtualMemory, NtProtectVirtualMemory
Context Read	NtGetContextThread
Thread State Control	NtSuspendThread, NtResumeThread, NtQueueApcThread
Process / Thread Exit	NtTerminateProcess, NtTerminateThread
Exception Handling	RtlAddVectoredExceptionHandler
System State / Sync	NtDelayExecution, NtCreateMutant
File Deletion	NtDeleteFile
Handle Close	NtClose
Decompression	RtlDecompressBuffer

そこで本研究では，Windows API の機能分類に関する文献 [8] などを参考に，表 1 に示す 17 種類のカテゴリを独自に定義した．以上より，各ログ行は「API 名・引数・カテゴリ」の 3 種類の情報を持つレコードに整形される．

3.2 特徴別画像化

動的解析ログからは，実行順序に基づく局所的な挙動と，全体的な出現頻度に基づく大局的な挙動の両面を捉えることが，未知の回避技術に対抗する上で有効であると考えられる．しかし，単一の画像化手法ではこれら双方を同時に適切に表現することは難しい．そこで本研究では，性質の異なる次の 2 種類の画像化手法を導入し，ログに含まれる特徴を多角的に表現する．

(1) **画像化手法 1**：順序情報（局所的依存関係）の画像化
(2) **画像化手法 2**：頻度情報（大局的傾向）の画像化

2 つの手法は互いに異なる性質の特徴を捉えるため，併用することで単独では捉えきれない多様な挙動パターンの補完が期待できる．以下，各手法を説明する．

3.2.1 画像化手法 1 (順序情報の画像化)

画像化手法 1 では，文献 [2] を参考に API・引数・カテゴリのうち 2 種類の要素を組み合わせ，局所的な遷移パターンを 2-gram として抽出する．処理手順を図 2(a) に示す．

まず，ログ中の時刻 t において選択した 2 要素（例：API と引数）を結合して 1 つの単語（Token）を生成し，これをログ全体に適用して単語列を得る．次に，得られた単語列から連続する 2 語の組（2-gram）を抽出する．2-gram の種類は膨大になるため，学習データ全体で集計したうえで χ^2 検定により分類への寄与度が高い上位 1,024 個を選択する．選択した 1,024 個を固定の並び順（Index 0〜Index 1023）に従って 32×32 画素の格子へ一対一にマッピングし，該当する 2-gram が解析対象のログ中に 1 回以上出現すればその画素を白（255），出現しなければ黒（0）とする

© 2026 Information Processing Society of Japan

二値画像を生成する．

このように画像化手法 1 は，API・引数・カテゴリの局所的な遷移パターンを固定の画素位置として表現し，局所的な実行パターンを視覚的特徴として抽出する．2-gram を単位とするのは，前後関係を適切な密度で捉えられ，かつ 3-gram 以上では組合せ数が急増して有意なパターンの密度が極端に低下するためである．文献 [2] においても，連続する 2 要素分に相当する長さの n-gram が採用されている．また，2-gram の有無を固定座標へマッピングするため，ダミー API の挿入などによりログが撹乱されても悪性の振る舞いを不変なパターンとして捉えることが可能となり，畳み込みカーネルが固定座標上に現れる 2-gram 群の局所的な共起関係を効率的に抽出できる．

3.2.2 画像化手法 2 (出現頻度の画像化)

画像化手法 2 では，文献 [3]，[4] を参考に API 名と引数を結合して 1 つの単語として扱い，ログ全体における単語の出現頻度を画像として表現する．処理概要を図 2(b) に示す．

まず，各 API 呼び出しについて API 名と引数を “LdrLoadDll_kernel32.dll” のように結合して単語列を生成し，ログ全体で各単語の出現回数を集計する．出現頻度が極端に低い単語はノイズとなるため，あらかじめ設定した基準に従って頻度の高い単語を優先的に選択する．その後，選択した単語を出現頻度の降順に並べ，頻度値を 0〜255 に正規化して 32×32 画素の画像の画素値として順に配置し，単語数が画素数より少ない場合はゼロパディングを行って固定サイズのグレースケール画像を生成する．高頻度単語に対応する画素は白く，ほとんど出現しない単語に対応する画素は黒く表現される．

このように画像化手法 2 は，順序情報などの局所的な遷移関係を考慮しない代わりに，ログ全体における API と引数の出現傾向を統計的特徴として強調する．生成画像は上部に高頻度成分の明るい領域，下部にゼロパディングの暗い領域が広がる構造を持ち，各検体の活動分布が大局的な形状として表現される．

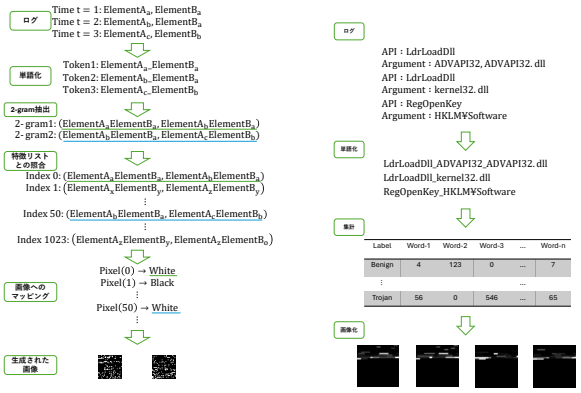
3.2.3 引数情報の結合方式

API 呼び出しには複数の引数が付与される場合があり，引数の扱い方によって単語化後の特徴表現が変化する．本研究では，Single 方式と Concat 方式の 2 種類を定義する．

Single 方式では，API（またはカテゴリ）と個々の引数を 1 対 1 で結合し，引数の数だけ単語を生成する．例えば，`NtCreateFile` が引数 `"virus.exe"` と `"Generic_Read"` を持つ場合，それぞれを個別に結合した `NtCreateFile_virus.exe`，`NtCreateFile_Generic_Read` の 2 語が得られる．各引数が独立した特徴として扱われるため，対象ファイルやアクセス権といった引数ごとの精細な挙動差を捉えられ，細粒度の分類に適している．

一方，Concat 方式では，API（またはカテゴリ）と全引数を順序通りに連結し，1 つの単語として扱う．上記の例では全引数を連結した `NtCreateFile_virus.exe_Generic_Read` の 1 語となる．引数全体が 1 単位の文脈として扱われるため，引数の組み合わせや API 呼び出し全体の意味的まとまりを保持した特徴表現が得られる．

Single 方式は細粒度の差異を，Concat 方式は文脈全体を表現するため，本研究では両者を併用している．



(a) 手法 1：順序情報の画像化　　(b) 手法 2：出現頻度の画像化

図 2 画像化手法の処理概要

Fig. 2 Process overview of Imaging Methods 1 and 2.

3.2.4 画像化パターンの組合せ

以上の処理により，3.1 で抽出した 3 要素に対して 2 種類の画像化手法と結合方式を適用し，計 7 種類の画像化パターンを生成する．画像化手法 1 では，API+Category，API+Argument（Single/Concat），Category+Argument（Single/Concat）の 5 種類，画像化手法 2 では API+Argument（Single/Concat）の 2 種類である．

API・引数・カテゴリの 3 要素を同時に結合すると組合せ数が急増し，$32 \times 32 = 1{,}024$ 画素に対して有意な特徴の密度が極端に低下するため，本研究では 2 要素の組合せに限定している．また，画像化手法 2 ではカテゴリを含む組合せを除外し「API+引数」に対象を絞っている．これは，画像化手法 1（順序）側でカテゴリによる抽象化表現を担う一方，大局的な出現頻度を扱う画像化手法 2 では，語彙数の少ないカテゴリを組み合わせると頻度分布が極端に一部の画素へ集中し情報密度が低下するためである．具体的には，抽象度の高いカテゴリを用いるよりも，精細な「API+引数」のレベルで全体的な出現分布を視覚化する方が，手法 2 の特徴強調に適していると判断したためである．

3.3 ベースモデルの学習と中間特徴量の取得

3.2 節で生成した各画像化パターンを，CNN をベースモデルとして個別に学習する．大規模な事前学習済みモデルを用いないのは，対象が自然画像ではなく 32×32 画素の単一チャネル（二値およびグレースケール）という低解像度なデータ構造を持つためである．そこで本研究では，文献 [2] で有効性が示された，畳み込み層とプーリング層を 2 組重ね

© 2026 Information Processing Society of Japan

る構造を参考に独自の軽量モデルを設計した．具体的には，32×32 の単一チャネル画像を入力とし，Conv2D(32, 5×5)–MaxPool(2×2)–Conv2D(64, 5×5)–MaxPool(2×2) で空間特徴を抽出した後，Flatten(4096)–Dense(256)–Dense(128)（いずれも ReLU）で中間表現へ圧縮する構成とした．各パターンを画像と良性・悪性の 2 値ラベルで学習し，出力層の確率値よりも情報を保ちやすい最終 Dense 層（128 次元）の特徴ベクトルを後段の統合に用いる．

全学習データに対しては，スタッキングで標準的な Out-of-Fold（OOF）により特徴を抽出する．5-fold で学習と抽出を分けることで，同一データを両方に用いるデータリークを避けられる．

3.4 特徴ベクトルの統合とメタモデルの学習

3.3 節で抽出した各ベースモデルの中間層特徴ベクトルを，スタッキングにより統合して良性・悪性を識別するメタモデルを構築する．

複数の画像パターンを統合する手法として，画像をチャネル方向に重ねて単一の CNN に入力するマルチチャネル入力も考えられる．しかし，本手法では局所的な順序関係（画像化手法 1）と大局的な出現頻度（画像化手法 2）という空間的性質の異なる情報を扱うため，これらをピクセルレベルで直接重ね合わせると，異質な特徴同士が干渉し，適切な学習が阻害される懸念がある．

そこで本手法ではスタッキングを採用し，各画像を独立した CNN で個別に学習させる構成とした．これにより，パターンごとに固有の特徴集合と画素配置に依存した中間特徴が抽出される．この独立して得られた表現の違いが相補性をもたらし，メタモデルで統合することで高精度かつロバストな検知を実現できる．

メタモデルには全結合ニューラルネットワークを用い，選択した各ベースモデルの中間層特徴（各 128 次元）を連結した $128 \times N$ 次元ベクトルを入力とする．ネットワーク構成は Dense(512)–Dense(256)–Dense(128)–Dense(128)（いずれも ReLU）–Output(1, Sigmoid) とし，対応するラベル（良性/悪性）を目的変数として学習を行う．これにより，各画像化パターンが抽出した特徴を総合的に利用した分類が可能となる．

4. 実験と考察

4.1 データセット

提案手法の有効性を確認するため，文献 [1] で公開されている大規模マルウェア動的解析ログデータセットを使用した．本研究では API 名，引数，カテゴリをすべて用いて画像化を行うため，解析ログ出力の形式不備により引数情報が欠落している検体を事前処理として除外した．

除外の結果，良性検体は 9,621 件から 8,372 件へ，マルウェア検体は 324,423 件から 320,594 件へと減少した．また，検体がすべて除外された 2 ファミリ（`asackytrojan`, `bancteian`）が存在したため，対象ファミリ数も 261 から 259 へ減少した．最終的なデータセットは良性 8,372 件，マルウェア 320,594 件の計 328,966 件であり，良性比率が約 2.5%の極端に不均衡な構成となっている．なお，本データセットには検体ごとにマルウェアファミリのラベルが付与されているが，本研究は良性・悪性の判定を目的とするため，良性以外のすべての検体を悪性として扱う 2 値ラベルに変換して用いる．

さらに，マルウェアのファミリ分布にも大きな偏りが存在し，一部のメジャーファミリに検体が集中している．このファミリ間不均衡が識別性能やモデルの安定性に与える影響については，4.6 節で詳細な検証を行う．

4.2 学習設定

4.2.1 提案手法の学習設定

極端なクラス不均衡に対応するため，通常の交差エントロピー誤差ではなく，少数クラスの損失重みを調整可能な Focal Loss[9] を損失関数として用いる．Focal Loss は以下で定義される．

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t) \tag{1}$$

ここで，p_t はモデルが正解クラスに対して出力した予測確率，α_t はクラス間の重みを調整するパラメータ，γ は分類が容易なサンプルの損失寄与を減衰させ，難解なサンプルに学習を集中させるフォーカシングパラメータである．本研究では $\gamma = 1.5$ とし，少数クラスである良性に対して $\alpha_t = 0.75$（多数クラスである悪性に対しては $1-\alpha_t = 0.25$）を設定した．ベースモデルの最適化には Adam（学習率 0.001）を用い，バッチサイズ 32，最大エポック数 50 とし，過学習を抑えるため Early Stopping を導入した．

メタモデルの入力には，3.3 節で述べた Out-of-Fold (OOF) 手法により未知データとして取得した各ベースモデルの中間層特徴（最終 Dense 層の 128 次元出力）を用いる．パターンごとに特徴量のスケールが異なるため，各特徴量を RobustScaler により正規化したうえで連結してメタモデルに入力する．メタモデルの最適化にも Adam（学習率 0.001）を用い，過学習抑制のため各全結合層の直後にドロップアウト層を挿入した．

4.2.2 1D 構成の学習設定（比較対象）

2 次元画像化および畳み込み（CNN）による空間特徴抽出の有効性を直接検証するため，比較対象として 1D 構成を構築した．1D 構成では，7 種類の画像化パターンそれぞれで選定した 1,024 個の特徴量を 1 次元ベクトルのまま個別の全結合ニューラルネットワークへ入力し，128 次元の中間表現へ圧縮する．中間特徴の次元数，スタッキングによる特徴統合の手順，メタモデルのアーキテクチャ，および損失関数や最適化手法といった学習設定はいずれも提案

© 2026 Information Processing Society of Japan

手法と完全に同一であり，両者の唯一の相違点は，特徴量を 2 次元へと空間配置して畳み込みを行うか否かという点にある．

4.3 評価指標

本研究では 3 つの実験を段階的に行う．実験 1 では各画像パターン単独の識別性能を確認し，実験 2 ではスタッキングによる統合の効果を検証する．さらに実験 3 では，ファミリ間のデータ不均衡に対するモデルの安定性を評価する．これら 3 つの実験は目的が異なるため，以下の評価指標を使い分ける．

(1) **実験 1・2：Benign F1-Score と PR-AUC**
実験 1 および実験 2 では，良性比率が約 2.5%の不均衡データを用いる．この条件では，すべての検体を悪性と判定する単純なモデルであっても高精度（Accuracy 約 97.5%）が算出されてしまうため，実運用における誤検知抑止を重視し，少数クラスである良性の識別に焦点を当てた Benign F1-Score を主指標とする．ただし，Benign F1-Score は判定閾値に依存するため，閾値に左右されない順位付け能力を評価する指標として PR-AUC（Precision-Recall Area Under the Curve）を併用する．PR-AUC は少数クラスの適合率（Precision）と再現率（Recall）に基づくため，不均衡データにおける識別性能をより正確に評価できる．

(2) **実験 3：Accuracy，Perfect Rate，Worst-Case Accuracy**
実験 3 では良性・悪性が 1:1 の均衡データを用いるため，全体的な分類性能を示す Accuracy を主指標とする．さらに，ファミリ間のデータ不均衡がモデルの安定性に与える影響を多角的に分析するため，本研究で定義する以下の 2 つの指標を併用する．ここで，$\mathcal{C}$ をテストデータに含まれるマルウェアファミリの集合，Acc_c をファミリ c における分類精度，$|\cdot|$ を集合の要素数とする．

$$\text{Perfect Rate} = \frac{|\{c \in \mathcal{C} \mid Acc_c = 1.0\}|}{|\mathcal{C}|} \tag{2}$$

$$\text{Worst-Case Accuracy} = \min_{c \in \mathcal{C}}(Acc_c) \tag{3}$$

Perfect Rate は分類精度が 100%となったファミリの割合であり，全検体を正解できたファミリが多いほど高い値を示す．Worst-Case Accuracy は全ファミリにおける分類精度の最小値であり，値が高いほどどのファミリに対しても安定して検知できていることを示す．これらを組み合わせることで，平均的な Accuracy だけでは隠蔽されやすい「多くのファミリは完璧に分類できるものの，一部のファミリを極端に苦手とする」といったモデルの偏りを客観的に評価できる．

表 2 各画像パターン単独の実験結果
Table 2 Experimental results of individual image-encoding patterns.

	画像化構成		1D 構成	
使用要素	Benign F1	PR-AUC	Benign F1	PR-AUC
従来手法 [1]	0.8631	—	—	—
画像化手法 *1*（局所的）				
API + Cat	0.8524	0.9417	0.8790	0.9397
API + Arg (Single)	**0.8674**	0.9330	0.8761	0.9477
API + Arg (Concat)	0.8615	0.9374	0.8702	0.9556
Cat + Arg (Single)	**0.8659**	0.9316	0.8788	0.9467
Cat + Arg (Concat)	0.8601	0.9312	0.8775	0.9460
画像化手法 *2*（大局的）				
API + Arg (Single)	0.8506	0.9102	0.8427	0.9090
API + Arg (Concat)	0.8370	0.9019	0.8454	0.9134

太字は画像化構成において従来手法を上回った値を示す．

4.4 実験 1：単独の画像化パターンの評価

本実験では，7 種類の画像化パターンをそれぞれ単独で用いた場合の識別性能を評価する．実験には 4.1 節で述べた不均衡データセットを用い，比較のため同一の特徴集合による 1D 構成についても同様に評価を行った．

4.4.1 実験結果 1 と考察

実験結果を表 2 に示す．各画像化手法を単独で評価した結果，画像化手法 1 において「API + Argument (Single)」が 0.8674，「Category + Argument (Single)」が 0.8659 の Benign F1-Score を記録し，従来手法 [1] を上回る精度を達成した．

この結果は，API の引数情報を画像化に組み込むことの有効性を示している．引数を用いない「API + Category」は 0.8524 と最も低い値を示しており，先行研究 [2] が主に API とカテゴリを用いていたのに対し，引数情報を組み込む方が高い精度を得られることが確認された．これは，API 名やカテゴリが操作の機能的性質を抽象化するのに対し，引数は操作対象のパスやアクセス権といった具体的な実行情報を保持するためと考えられる．

一方で，画像化手法 2（頻度情報）単独ではいずれの設定も比較手法の精度に及ばず，PR-AUC においても画像化手法 1 を明確に下回った．これは，時系列的なコンテキストを考慮せず統計的な頻度分布のみを特徴量として抽出したため，順序依存的な攻撃パターンを十分に捉えきれなかったためと考えられる．

また 1D 構成との比較では，画像化手法 2 の Single 方式を除く 6 種類において 1D 構成が画像化構成を上回った．このように単体での優劣はパターンや手法の組合せにより一様ではなく，単一の画像表現のみでは 2 次元配置と畳み込みの優位性を安定的に発揮することが難しい．ただし，本実験は各パターン単体での基礎性能を確認したものであり，提案手法の主眼は性質の異なる複数表現の統合にある．次節では，これらを統合した際の相補的効果について検証

© 2026 Information Processing Society of Japan

表 3 スタッキングの組合せと評価結果

Table 3 Stacking combinations and evaluation results.

	画像化手法 1					画像化手法 2		画像化構成		1D 構成	
特徴数	API +Cat	API+Arg (Sin.)	API+Arg (Con.)	Cat+Arg (Sin.)	Cat+Arg (Con.)	API+Arg (Sin.)	API+Arg (Con.)	Benign F1	PR-AUC	Benign F1	PR-AUC
比較手法 [1]	—	—	—	—	—	—	—	0.8631	—	—	—
2 種類	●	—	—	●	—	—	—	0.9022	0.9576	0.8875	0.9544
3 種類	●	●	—	●	—	—	—	0.9026	0.9583	0.8836	0.9547
4 種類	●	●	—	●	●	—	—	0.9042	0.9581	0.8892	0.9547
5 種類	●	●	—	●	●	●	—	**0.9056**	**0.9600**	0.8912	0.9555
6 種類	●	●	—	●	●	●	●	0.9030	0.9575	0.8925	0.9566
7 種類	●	●	●	●	●	●	●	0.8845	0.9540	**0.8967**	**0.9579**

● は当該パターンを統合に用いたことを示す．Sin. は Single 方式，Con. は Concat 方式を表す．各特徴数において最も高い Benign F1-Score を示した組合せを掲載した．太字は画像化構成・1D 構成それぞれにおける最高値を示す．

する．

4.5 実験 2：スタッキングによる統合の評価

実験 1 の結果を踏まえ，単独の画像パターンに依存するのではなく，異なる特徴を持つ複数の画像パターンをスタッキングして統合した際，モデルの識別性能にどのような変化が生じるかを検証した．実験設定には実験 1 と同様に不均衡データセットを用いた．画像パターンの組み合わせ数は 2 種類から 7 種類まで変化させ，各特徴数において最も高い精度を示した組み合わせを評価した．また実験 1 と同様に，1D 構成についても同一の手順でスタッキングを行い比較した．

4.5.1 実験結果 2 と考察

実験結果を表 3 に示す．単独の画像パターンでは最高 0.8674 であった Benign F1-Score がスタッキングにより大幅に向上し，特徴量を 5 種類組み合わせた構成において最高値の 0.9056 を達成した．PR-AUC においても同構成が 0.9600 で最高値を示している．この最良構成は，画像化手法 1（順序）から 4 種，画像化手法 2（頻度）から 1 種で構成されている．手法 1 の 4 特徴のみの統合でも 0.9042 に達しており，時系列コンテキストを主軸としつつ，手法 2 の頻度情報を 1 つ融合させることで識別境界が適正に補正されたと考えられる．一方，特徴量を 6 種類以上に増やすと，類似したパターンの重複に伴う冗長な特徴の混入により，メタモデルが過適合（過学習）を起こし汎化性能が低下した．

1D 構成との比較では，組み合わせ数が 2 から 6 のすべてにおいて画像化構成が 1D 構成を明確に上回った．特に画像化構成は 2 種類を統合した時点で 0.9022 に達し，1D 構成がすべてを統合した場合の最高値（0.8967）を上回っている．この差は，畳み込み抽出特徴が持つ表現の多様性に起因すると考えられる．1D 構成の全結合層は重複の多いパターンに対して類似表現へ収束しやすいのに対し，CNN は局所受容野の制約によりパターンごとに固有の中間表現を保持するため，スタッキング統合時の相補性を最大化できたと推察される．

以上より，精度向上には無差別にパターンを増やすのではなく，実行順序による局所的文脈をベースとし，出現頻度の大局的傾向を過学習を招かない粒度で相補的に加える特徴厳選が重要であることが示された．次節では，この最良構成を用いてファミリ間のデータ不均衡に対するモデルの安定性を検証する．

4.6 実験 3：ファミリ間不均衡に対する安定性

実験 2 で最も高い精度を示した 5 種類の組み合わせを用い，ファミリ間のデータ不均衡に対するモデルの安定性を検証する．なお，良性検体数に合わせて悪性検体全体からのランダムサンプリングにより構築した 1:1 の均衡データセット（良性 8,372 件，悪性 8,372 件）において評価を行ったところ，5 回の試行で平均 Accuracy 0.9701 を記録した．以下の実験では，この均衡条件を基盤として，ファミリごとの学習データ数の偏りがモデルの安定性に与える影響を詳細に検証する．

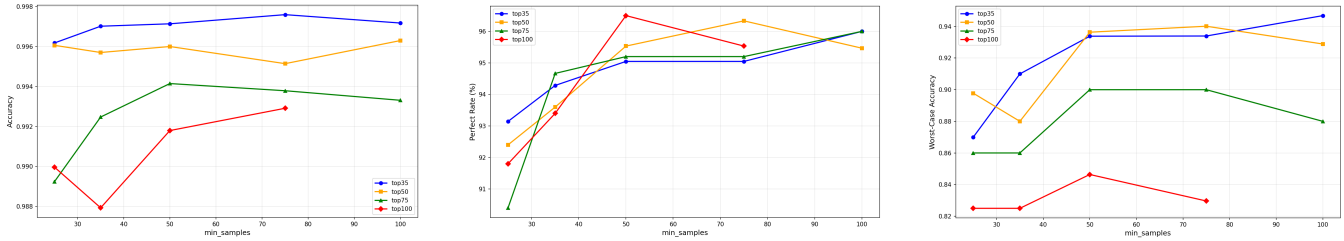
完全ランダムサンプリングでは検体数の多いメジャーファミリがデータセットの大半を占めるため，実運用で重要なマイナーファミリへの対応能力を十分に評価できない課題が残る．そこで，データの偏りに依存しないモデル本来の安定性を検証するため，各ファミリの最低学習サンプル数（25〜100 件）と対象ファミリ数（35〜100）を段階的に制御した実験を行った．

4.6.1 実験結果 3 と考察

実験結果を図 3 に示す．1 ファミリあたりの最低学習サンプル数を 25 件から 50 件へ増加させるにつれて，Accuracy，Worst-Case Accuracy，Perfect Rate のいずれの指標においても顕著な改善が観測された．Worst-Case Accuracy の向上は識別精度の低かったマイナーファミリの検知能力が改善したことを示し，Perfect Rate の向上は試行間のバラつきが減少し安定して検知できるファミリが増加したことを反映している．

また，本手法の安定性に対しては，対象ファミリ数の多

© 2026 Information Processing Society of Japan



(a) Accuracy (b) Perfect Rate (c) Worst-Case Accuracy

図 3 学習データ数およびファミリ数の変化に伴う各指標の推移

Fig. 3 Transitions of evaluation metrics with varying training samples and families.

寡よりも各ファミリにおける最低学習サンプル数の確保が大きく影響することが示された．1 ファミリあたり最低 50 件を確保した条件では，対象ファミリ数を 100 まで拡大しても高い性能が維持・向上する傾向が確認された．ただし，サンプル数を 75 件や 100 件と過剰に増やすと，メタモデルが特定ファミリ固有のノイズ表現に過適合し，Worst-Case Accuracy がやや低下する傾向も見られた．

以上より，1 ファミリあたり 50 件程度の学習サンプルを確保すれば，100 種類規模の広範なファミリに対しても高い Accuracy と検知安定性を維持でき，未知のファミリを新たに追加学習する際も最小限の検体を収集・処理すれば十分対応可能であることが示唆される．

5. おわりに

本研究では，動的解析ログにおける API 名・引数・カテゴリの 3 要素を包括的に活用し，複数の画像化手法と CNN，スタッキングを組み合わせた 2 値分類によるマルウェア検知手法を提案した．局所的な順序関係と大局的な出現頻度という異なる性質の画像表現を統合することにより，良性比率約 2.5%の不均衡データにおいて従来手法を上回る検知精度を達成した．さらに，1 ファミリあたり 50 件程度の学習サンプルを確保すれば，100 種類規模の多様なファミリに対しても高い検出安定性を維持できることを確認した．

今後は，より高度な空間表現や大局的コンテキストを抽出可能な最新の画像識別モデル（Vision Transformer 等）の適用や，未知の複雑な順序撹乱技術に対する耐性の詳細な評価を課題とする．また，本研究では空間的性質の異なる特徴間の干渉を防ぐ目的でスタッキングを採用したが，マルチチャネル入力などの異なる統合アーキテクチャとの定量的比較や，特徴表現間の相補・干渉メカニズムに関するより詳細な解析を進める予定である．

参考文献

[1] Fellicious, C., Bischof, M., Mayer, K., et al.: Malware Detection based on API calls, arXiv preprint arXiv:2502.12863 (2025).

[2] Shiva Darshan, S. L. and Jaidhar, C. D.: Windows Malware Detector Using Convolutional Neural Network Based on Visualization Images, *IEEE Transactions on Emerging Topics in Computing*, Vol.9, No.2, pp.1057–1069 (2021).

[3] Shahnawaz, M., Gond, B. P. and Mohapatra, D. P.: Dynamic Malware Classification of Windows PE Files using CNNs and Greyscale Images Derived from Runtime API Call Argument Conversion, *Proc. 16th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (2025).

[4] Gond, B. P., Singh, A. K. and Mohapatra, D. P.: A Deep Learning Framework for Malware Classification using NLP Techniques, *Proc. 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp.426–433 (2024).

[5] Damaševičius, R., Venčkauskas, A., Toldinas, J., et al.: Ensemble-Based Classification Using Neural Networks and Machine Learning Models for Windows PE Malware Detection, *Electronics*, Vol.10, No.4, 485 (2021).

[6] Roy, K. S., Ahmed, T., Udas, P. B., et al.: MalHyStack: A Hybrid Stacked Ensemble Learning Framework with Feature Engineering Schemes for Obfuscated Malware Analysis, *Intelligent Systems with Applications*, Vol.20, 200283 (2023).

[7] Lakshmanarao, A. and Shashi, M.: An Efficient Android Malware Detection Framework with Stacking Ensemble Model, *International Journal of Engineering Trends and Technology*, Vol.70, No.4, pp.294–302 (2022).

[8] Gupta, S., Sharma, H. and Kaur, S.: Malware Characterization Using Windows API Call Sequences, *Journal of Cyber Security and Mobility*, Vol.7, No.4, pp.231–246 (2018).

[9] Lin, T.-Y., Goyal, P., Girshick, R., et al.: Focal Loss for Dense Object Detection, *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp.2980–2988 (2017).

© 2026 Information Processing Society of Japan